

COMPLETE REFERENCE



SQL PRO

2026 Cheatsheet

Every essential SQL concept — from SELECT basics to window functions, CTEs, and JSON — with plain-English explanations and real query examples with output.

20 Sections

90+ Queries

Real Output



@MohitDecodes

Table of Contents

Organized from the absolute basics to advanced modern SQL — study in order, or jump straight to the section you need.

01	Basic SQL Syntax SELECT, WHERE, ORDER BY, DISTINCT, LIMIT	5
02	Filtering & Conditions AND/OR, LIKE, BETWEEN, IN, NULL checks	5
03	Aggregate Functions COUNT, SUM, AVG, MIN, MAX	5
04	GROUP BY & HAVING Grouping rows and filtering groups	3
05	SQL Joins INNER, LEFT, RIGHT, FULL, SELF, CROSS	6
06	Subqueries & Aliases Scalar, correlated, EXISTS, AS	4
07	CASE Statement Conditional logic inside queries	2
08	Window Functions RANK, ROW_NUMBER, LAG/LEAD, running totals	6
09	String Functions UPPER, LOWER, LENGTH, SUBSTRING, CONCAT, TRIM	6
10	Date & Time Functions CURRENT_DATE, NOW, DATE_ADD, DATEDIFF, EXTRACT	5
11	Data Types Numeric, string, date, boolean, JSON	3
12	Table Operations (DDL) CREATE, ALTER, DROP, TRUNCATE, RENAME	5
13	Insert, Update, Delete (DML) INSERT, UPDATE, DELETE, UPSERT	4
14	Constraints PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK	4
15	Indexes & Optimization CREATE INDEX, EXPLAIN, tuning tips	4
16	Views CREATE VIEW, materialized views	3
17	Stored Procedures & Functions Reusable server-side logic, triggers	3
18	Transactions BEGIN, COMMIT, ROLLBACK, SAVEPOINT	4
19	CTEs & Set Operators WITH, recursive CTE, UNION, INTERSECT, EXCEPT	5
20	JSON & Modern SQL JSON functions, ARRAY_AGG, STRING_AGG	4

SELECT

Retrieves specific columns from a table. Use * to select every column.

```
SELECT name, salary FROM employees;
```

```
name | salary  
-----+-----  
Avani | 75000  
Rahul | 62000
```

WHERE

Filters rows so only those matching a condition are returned.

```
SELECT * FROM employees WHERE salary > 50000;  
-- Returns only employees earning above 50,000
```

ORDER BY

Sorts the result set. **ASC** is ascending (default), **DESC** is descending.

```
SELECT * FROM employees ORDER BY salary DESC;  
-- Highest paid employees appear first
```

DISTINCT

Removes duplicate values from the results.

```
SELECT DISTINCT department  
FROM employees;
```

```
HR  
Sales  
Engineering
```

LIMIT / OFFSET

Restricts how many rows return — the basis of pagination.

```
SELECT * FROM employees  
LIMIT 10 OFFSET 20;  
-- Rows 21-30 (page 3)
```

Execution order tip: SQL runs clauses in this order — FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. That's why you can't use a SELECT alias inside WHERE, but you can inside ORDER BY.

Filtering & Conditions

Narrowing results down to exactly the rows you want

AND / OR

Combine multiple conditions in one filter.

```
SELECT * FROM employees
WHERE salary > 50000
AND department = 'HR';
```

LIKE

Pattern matching. % = any characters, _ = one character.

```
SELECT * FROM users
WHERE name LIKE 'A%';
-- Names starting with A
```

BETWEEN

Matches a value inside an inclusive range.

```
SELECT * FROM users
WHERE age BETWEEN 18 AND 25;
```

IN

Matches any value from a given list.

```
SELECT * FROM customers
WHERE country IN ('India', 'USA');
```

IS NULL / IS NOT NULL

Checks for missing values. You must use **IS NULL** — comparing with **= NULL** never works, because NULL is not equal to anything, not even itself.

```
SELECT * FROM users WHERE phone IS NULL;
-- Wrong: WHERE phone = NULL (returns nothing!)
```

Aggregate Functions

Summarizing many rows into a single value

COUNT / SUM / AVG / MIN / MAX

Compute totals and statistics across a set of rows.

```
SELECT COUNT(*) FROM orders; -- 1,204
SELECT SUM(amount) FROM sales; -- 845000
SELECT AVG(salary) FROM employees; -- 68500
SELECT MIN(price), MAX(price) FROM products;
```

Watch out: COUNT(*) counts every row including NULLs, but COUNT(column) skips rows where that column is NULL. AVG and SUM also ignore NULL values entirely.

GROUP BY

Collapses rows into groups so aggregates are calculated per group.

```
SELECT department, COUNT(*) AS total
FROM employees GROUP BY department;
```

```
department | total
-----+-----
HR | 12
Engineering | 48
```

HAVING

Filters **groups** after aggregation — WHERE filters rows **before** it.

```
SELECT department, COUNT(*) FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
-- Only departments with more than 5 people
```

WHERE vs HAVING: WHERE filters individual rows before grouping and cannot use aggregate functions. HAVING filters the grouped results afterwards and is where COUNT, SUM and AVG conditions belong.

JOIN TYPE	WHAT IT RETURNS
INNER	Only rows with a match in both tables
LEFT	All rows from the left table + matched right (NULL if none)
RIGHT	All rows from the right table + matched left
FULL OUTER	All rows from both tables, matched and unmatched
CROSS	Every combination of rows (Cartesian product)
SELF	A table joined to itself, using aliases

INNER JOIN

Returns only orders that have a matching customer.

```
SELECT o.id, c.name FROM orders o
INNER JOIN customers c ON o.customer_id = c.id;
```

LEFT JOIN

Keeps every customer, even those who never placed an order.

```
SELECT c.name, o.id FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id;
```

```
name | id
-----+-----
Avani | 1001
Rahul | NULL -- no orders yet
```

SELF JOIN

Joins a table to itself — here, matching each employee to their manager.

```
SELECT e.name, m.name AS manager
FROM employees e
JOIN employees m
ON e.manager_id = m.id;
```

CROSS JOIN

Pairs every row with every other row — use with care.

```
SELECT s.size, c.color
FROM sizes s
CROSS JOIN colors c;
-- 3 sizes x 4 colors = 12 rows
```

Join gotcha: Putting a condition on the right table in WHERE instead of ON silently turns a LEFT JOIN into an INNER JOIN, because NULL rows get filtered out. Keep right-table filters inside the ON clause.

Scalar Subquery

An inner query that returns a single value, used in a comparison.

```
SELECT name FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
-- Everyone earning above the company average
```

EXISTS

Checks whether the subquery returns any rows at all — often faster than IN for large datasets.

```
SELECT c.name FROM customers c
WHERE EXISTS (
  SELECT 1 FROM orders o WHERE o.customer_id = c.id
);
-- Customers who have placed at least one order
```

Column Alias (AS)

Renames a column in the output.

```
SELECT name AS EmployeeName
FROM users;
```

Table Alias

Shortens table names in joins.

```
SELECT e.name, d.title
FROM employees e
JOIN departments d
ON e.dept_id = d.id;
```

CASE WHEN ... THEN ... ELSE ... END

Returns a different value per row depending on conditions.

```
SELECT name,  
CASE  
WHEN score >= 90 THEN 'A'  
WHEN score >= 80 THEN 'B'  
ELSE 'C'  
END AS grade  
FROM students;
```

```
name | grade  
-----+-----  
Avani | A  
Rahul | B
```

CASE inside an aggregate

A powerful trick — conditional counting in a single pass.

```
SELECT  
COUNT(CASE WHEN status = 'paid' THEN 1 END) AS paid,  
COUNT(CASE WHEN status = 'due' THEN 1 END) AS due  
FROM invoices;
```

RANK() OVER (ORDER BY ...)

Assigns a rank to each row. Ties share a rank, and the next rank skips ahead.

```
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS rank
FROM employees;
```

```
name | salary | rank
-----+-----+-----
Avani | 95000  | 1
Rahul | 95000  | 1
Neha  | 80000  | 3 -- 2 is skipped
```

ROW_NUMBER() OVER (PARTITION BY ...)

Numbers rows 1,2,3... restarting the count within each partition.

```
SELECT *, ROW_NUMBER() OVER (
  PARTITION BY department ORDER BY salary DESC
) AS rn
FROM employees;
-- rn = 1 is the top earner in each department
```

DENSE_RANK()

Like RANK, but never skips numbers after a tie.

```
DENSE_RANK() OVER (
  ORDER BY salary DESC
-- 1, 1, 2 (not 1, 1, 3)
```

NTILE(n)

Splits rows into n roughly equal buckets.

```
NTILE(4) OVER (
  ORDER BY salary
-- salary quartiles 1-4
```

LAG() / LEAD()

Access the previous or next row's value — ideal for month-over-month comparisons.

```
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_month
FROM sales;
```

```
month | revenue | prev_month
-----+-----+-----
Jan   | 50000   | NULL
Feb   | 62000   | 50000
```

Why windows matter: Unlike GROUP BY, a window function keeps every row intact while still giving you the aggregate alongside it — so you can show each salary *and* its rank in the same result.

Running Total

A cumulative sum built with a window frame — each row adds to the previous.

```
SELECT month, revenue,
SUM(revenue) OVER (ORDER BY month) AS running_total
FROM sales;
```

```
month | revenue | running_total
-----+-----+-----
Jan   | 50000   | 50000
Feb   | 62000   | 112000
Mar   | 48000   | 160000
```

UPPER / LOWER

Converts text case.

```
SELECT UPPER(name) FROM users;
-- 'avani' → 'AVANI'
```

LENGTH

Returns number of characters.

```
SELECT LENGTH(name) FROM users;
-- 'Avani' → 5
```

SUBSTRING

Extracts part of a string.

```
SUBSTRING(name, 1, 3)
-- 'Avani' → 'Ava'
```

CONCAT

Joins strings together.

```
CONCAT(first, ' ', last)
-- 'Avani Sharma'
```

TRIM

Removes leading/trailing spaces.

```
TRIM(' hello ')
-- 'hello'
```

REPLACE

Swaps one substring for another.

```
REPLACE(email, '.com', '.in')
```

CURRENT_DATE / NOW()

Returns today's date, or the full current timestamp.

```
SELECT CURRENT_DATE; -- 2026-01-15
SELECT NOW(); -- 2026-01-15 14:32:07
```

DATE_ADD

Adds an interval to a date.

```
DATE_ADD(order_date,
INTERVAL 7 DAY)
```

DATEDIFF

Days between two dates.

```
DATEDIFF(end_date, start_date)
-- 14
```

EXTRACT

Pulls a single component — year, month, day — out of a date.

```
SELECT EXTRACT(YEAR FROM birth_date) AS birth_year
FROM users; -- 1998
```

DATE_TRUNC

Rounds a timestamp down — perfect for monthly reports.

```
SELECT DATE_TRUNC('month', created_at)
AS month, COUNT(*)
FROM orders GROUP BY month;
```

Age / Interval math

Subtract dates directly to get an interval.

```
SELECT CURRENT_DATE - birth_date
AS days_old
FROM users;
```

Filtering by date range

The most common real-world date query — everything in a given period.

```
SELECT * FROM orders
WHERE order_date >= '2026-01-01'
AND order_date < '2026-02-01';

-- Avoid: WHERE EXTRACT(MONTH FROM order_date) = 1
-- Wrapping the column in a function disables the index!
```

Dialect note: Date functions vary the most between databases. MySQL uses `DATE_ADD` and `DATEDIFF`, PostgreSQL prefers `date + INTERVAL '7 days'` and `AGE()`, and SQL Server uses `DATEADD/DATEDIFF`. Always check your engine's docs.

CATEGORY	COMMON TYPES
Numeric	INT, BIGINT, FLOAT, DECIMAL(10,2) — use DECIMAL for money
Text	VARCHAR(255), CHAR(n), TEXT
Date/Time	DATE, TIME, DATETIME, TIMESTAMP
Other	BOOLEAN, JSON, UUID, BLOB

Types in practice

Pick the smallest type that safely fits your data — it saves storage and speeds up queries.

```
CREATE TABLE users (  
  id INT,  
  name VARCHAR(100),  
  balance DECIMAL(10,2),  
  is_active BOOLEAN  
);
```

CREATE TABLE

Defines a new table and its columns.

```
CREATE TABLE employees (  
  id INT PRIMARY KEY,  
  name TEXT NOT NULL,  
  salary INT  
);
```

ALTER TABLE ... ADD

Adds a new column.

```
ALTER TABLE employees  
ADD COLUMN salary INT;
```

ALTER TABLE ... RENAME

Renames a table.

```
ALTER TABLE employees  
RENAME TO staff;
```

DROP TABLE

Deletes the table and all its data.

```
DROP TABLE employees;  
-- Irreversible!
```

TRUNCATE TABLE

Empties all rows but keeps the table.

```
TRUNCATE TABLE logs;  
-- Faster than DELETE
```

DROP vs TRUNCATE vs DELETE: DROP removes the table entirely. TRUNCATE removes all rows quickly but keeps the structure. DELETE removes rows one by one and can be filtered with WHERE and rolled back in a transaction.

INSERT INTO

Adds one or many new rows.

```
INSERT INTO users (name, age) VALUES ('Avani', 25);

-- Multiple rows at once
INSERT INTO users (name, age)
VALUES ('Rahul', 30), ('Neha', 28);
```

UPDATE ... SET

Modifies existing rows. Always include a WHERE clause.

```
UPDATE users SET age = 26 WHERE name = 'Avani';
-- Without WHERE, EVERY row is updated!
```

DELETE FROM

Removes matching rows.

```
DELETE FROM users
WHERE age < 20;
```

UPSERT

Insert, or update if it already exists.

```
INSERT INTO users (id, name)
VALUES (1, 'Avani')
ON CONFLICT (id) DO UPDATE
SET name = EXCLUDED.name;
```

Safety habit: Before running an UPDATE or DELETE, run the same WHERE clause as a SELECT first. If the row count looks wrong, your filter is wrong — and you just avoided a very bad day.

CONSTRAINT	WHAT IT ENFORCES
PRIMARY KEY	Unique + NOT NULL — identifies each row
FOREIGN KEY	Value must exist in another table
UNIQUE	No duplicate values in the column
NOT NULL	Column can never be empty
DEFAULT	Value used when none is supplied
CHECK	Value must satisfy a condition

Constraints in a CREATE TABLE

Applied at column level when the table is defined.

```
CREATE TABLE orders (
  id INT PRIMARY KEY,
  customer_id INT REFERENCES customers(id),
  amount FLOAT CHECK (amount > 0),
  status VARCHAR(20) DEFAULT 'pending'
);
```

Add a constraint later

Apply rules to an existing table.

```
ALTER TABLE users
ADD CONSTRAINT uq_email
UNIQUE (email);
```

ON DELETE CASCADE

Auto-delete child rows with the parent.

```
customer_id INT
REFERENCES customers(id)
ON DELETE CASCADE
```

Why constraints matter: Validation in application code can be bypassed by a script, a migration, or another service. Constraints live in the database itself, so bad data simply cannot be written — no matter who writes it.

CREATE INDEX

Speeds up lookups on a column — at the cost of slightly slower writes.

```
CREATE INDEX idx_name ON employees(name);

-- Composite index for multi-column filters
CREATE INDEX idx_dept_sal ON employees(department, salary);
```

DROP INDEX

Removes an unused index.

```
DROP INDEX idx_name;
```

UNIQUE INDEX

Index that also blocks duplicates.

```
CREATE UNIQUE INDEX idx_email
ON users(email);
```

EXPLAIN

Shows the query plan so you can see whether an index is actually used.

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 5;

Index Scan using idx_customer on orders
(cost=0.29..8.31 rows=1) -- good!
Seq Scan on orders (cost=0.00..431.00) -- needs an index
```

Optimization checklist: Index columns used in WHERE and JOIN; avoid SELECT * in production; filter early; don't wrap indexed columns in functions (WHERE YEAR(date)=2026 skips the index); and check EXPLAIN before assuming a query is fast.

CREATE VIEW

Stores a query under a name — it runs fresh each time you select from it.

```
CREATE VIEW active_users AS
SELECT * FROM users WHERE is_active = TRUE;

SELECT * FROM active_users; -- query it like a table
```

DROP VIEW

Removes a view (data is untouched).

```
DROP VIEW active_users;
```

MATERIALIZED VIEW

Stores results physically — fast, but needs refreshing.

```
CREATE MATERIALIZED VIEW mv_sales
AS SELECT ...;
REFRESH MATERIALIZED VIEW mv_sales;
```

CREATE PROCEDURE

A named block of SQL you can call repeatedly.

```
CREATE PROCEDURE GetTotalOrders()
BEGIN
SELECT COUNT(*) FROM orders;
END;

CALL GetTotalOrders();
```

CREATE FUNCTION

Like a procedure, but returns a value and can be used inside a query.

```
CREATE FUNCTION get_bonus(salary INT)
RETURNS INT
RETURN salary * 0.10;

SELECT name, get_bonus(salary) FROM employees;
```

TRIGGER

Runs automatically when a table is modified — useful for audit logs.

```
CREATE TRIGGER log_update
AFTER UPDATE ON employees
FOR EACH ROW
INSERT INTO audit_log(emp_id, changed_at)
VALUES (NEW.id, NOW());
```

BEGIN / COMMIT / ROLLBACK

Either every statement succeeds together, or none of them apply.

```
BEGIN;  
UPDATE accounts SET balance = balance - 500 WHERE id = 1;  
UPDATE accounts SET balance = balance + 500 WHERE id = 2;  
COMMIT; -- save both changes  
-- ROLLBACK; would undo everything instead
```

SAVEPOINT

A checkpoint you can roll back to without abandoning the whole transaction.

```
BEGIN;  
SAVEPOINT beforeUpdate;  
UPDATE users SET age = 30;  
ROLLBACK TO beforeUpdate; -- undo just that step  
COMMIT;
```

ACID in one line: Atomicity (all or nothing), Consistency (rules always hold), Isolation (concurrent transactions don't corrupt each other), Durability (committed data survives a crash).

WITH ... AS (CTE)

A named temporary result set that makes complex queries far easier to read.

```
WITH high_earners AS (  
  SELECT name, salary FROM employees WHERE salary > 100000  
)  
SELECT * FROM high_earners ORDER BY salary DESC;
```

RECURSIVE CTE

A CTE that references itself — used for hierarchies like org charts or category trees.

```
WITH RECURSIVE subordinates AS (  
  SELECT id, name, manager_id FROM employees WHERE id = 1  
  UNION ALL  
  SELECT e.id, e.name, e.manager_id FROM employees e  
  JOIN subordinates s ON e.manager_id = s.id  
)  
SELECT * FROM subordinates;
```

OPERATOR	WHAT IT RETURNS
UNION	Rows from both queries, duplicates removed
UNION ALL	Rows from both queries, duplicates kept (faster)
INTERSECT	Only rows present in both queries
EXCEPT	Rows in the first query but not the second

Set operators in action

Both queries must return the same number and type of columns.

```
SELECT name FROM table1 UNION SELECT name FROM table2;  
SELECT name FROM table1 INTERSECT SELECT name FROM table2;  
SELECT name FROM table1 EXCEPT SELECT name FROM table2;
```

JSON_EXTRACT

Reads a field from a JSON column.

```
SELECT JSON_EXTRACT(data, '$.name')  
FROM users;
```

ARRAY_AGG

Collapses a column into an array.

```
SELECT ARRAY_AGG(name)  
FROM employees;  
-- {Avani, Rahul, Neha}
```

STRING_AGG / GROUP_CONCAT

Joins values from many rows into one delimited string.

```
SELECT department, STRING_AGG(name, ', ') AS team  
FROM employees GROUP BY department;
```

```
department | team  
-----+-----  
HR | Avani, Rahul, Neha
```

JSON column type

Store flexible, schema-less data in a relational table.

```
CREATE TABLE events (  
  id INT,  
  payload JSON  
);
```

COALESCE

Returns the first non-NULL value — great for defaults.

```
SELECT COALESCE(nickname,  
  name, 'Guest') FROM users;
```

Use JSON sparingly: It's excellent for genuinely variable data like event payloads or settings. But if you find yourself querying the same JSON key constantly, that key deserves to be a real, indexable column.



Practice Challenges

Test yourself — if you can write these, you know SQL

Challenge 1 – Second highest salary

A classic interview question. Window functions make it clean.

```
SELECT DISTINCT salary FROM (  
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) r  
  FROM employees  
 ) t WHERE r = 2;
```

Challenge 2 – Find duplicate rows

Group by the column, then keep groups appearing more than once.

```
SELECT email, COUNT(*) FROM users  
GROUP BY email  
HAVING COUNT(*) > 1;
```

Challenge 3 – Customers with no orders

A LEFT JOIN plus a NULL check — no subquery needed.

```
SELECT c.name FROM customers c  
LEFT JOIN orders o ON c.id = o.customer_id  
WHERE o.id IS NULL;
```

Challenge 4 – Top earner per department

PARTITION BY restarts the ranking inside each group.

```
SELECT * FROM (  
  SELECT *, ROW_NUMBER() OVER (  
    PARTITION BY department ORDER BY salary DESC) rn  
  FROM employees  
 ) t WHERE rn = 1;
```

How to keep improving: Write queries against a real dataset, always run EXPLAIN on anything slow, and get comfortable with window functions and CTEs — they're what separate intermediate SQL from genuinely advanced SQL.